

Trabajo Práctico Integrador

Food Store

Sistema de Gestión de Pedidos Gastronómicos — Java 21

Alfredo Castillo · Comisión 3

Cesia Castilla · Comisión 5

Rocío Agüero · Comisión 6

Arquitectura en capas · Persistencia en memoria · Sin login (rol operador)

Objetivo del Proyecto

Qué es Food Store y cómo está planteado según la consigna

ACCESO

Sin login

Acceso directo para un único rol: **operador**. No requiere autenticación.

PERSISTENCIA

En memoria

Toda la información se maneja con **colecciones** (ArrayList), sin base de datos.

TECNOLOGÍA

Java 21

Desarrollado y ejecutable desde **NetBeans** o por **terminal**.

Equipo de 3 integrantes: cada uno estuvo a cargo de módulos distintos del sistema — Categorías y arquitectura general (Alfredo), Productos y Usuarios (Cesia), Pedidos (Rocío).

Arquitectura en Capas

Organización por paquetes y punto de entrada de la aplicación



Repositorio Genérico y Baja Lógica

CategoriaRepository.java — interfaz IRepository<T>

Operación	Método	Comportamiento
Listar	<code>listarTodos()</code>	Devuelve solo registros con <code>eliminado = false</code>
Buscar	<code>buscarPorId()</code>	Filtra también por el flag de eliminado
Eliminar	<code>eliminar()</code>	No remueve físicamente: <code>categoria.setEliminado(true)</code>

Baja Lógica (Soft Delete): exigida por la cátedra. En vez de borrar el objeto del ArrayList —lo que rompería el historial si hay productos o pedidos asociados— se cambia su estado. El registro queda oculto para el operador pero salvaguardado en memoria.

Reglas de Negocio y Excepciones

CategoriaService.java

- **validarNombre():** arroja excepción si el nombre está vacío o si ya existe otra categoría con ese nombre (comparación sin distinguir mayúsculas/minúsculas).
- **EntidadNoEncontradaException:** excepción personalizada en tiempo de ejecución, disparada al buscar o dar de baja un ID inexistente — asegura un manejo controlado de errores.
- **tieneProductosAsociados():** informa al operador si la categoría que se está alterando posee stock vinculado, sin bloquear el flujo del sistema.

Idea central: el Service controla que se cumplan las reglas comerciales antes de impactar el repositorio.

Gestión de Productos

Producto · ProductoRepository · ProductoService · MenuProducto

ENTIDAD

- Hereda de `Base` : nombre, precio, descripción, stock, imagen, disponibilidad.
- Referencia directa a `Categoria` — relación de objeto real **unidireccional**: el Producto conoce su Categoría, pero la Categoría no tiene una lista de Productos.

VALIDACIONES

- Nombre no vacío.
- Precio y stock no negativos → `StockInvalidoException` / `ValidationException` .
- Categoría existente y no eliminada → `EntidadNoEncontradaException` .

MENÚ

- Listar (con filtro opcional por categoría), crear, editar.
- Eliminar con **baja lógica**: nunca se borra físicamente porque puede estar referenciado en detalles de pedidos existentes.
- Confirmación explícita (S/N) antes de eliminar.
- Lecturas numéricas (precio, stock, IDs) reintentan ante entrada inválida en vez de romper el programa.

Gestión de Usuarios

Usuario · UsuarioRepository · UsuarioService · MenuUsuario

ENTIDAD

- Hereda de `Base` : nombre, apellido, mail, celular, contraseña y rol (`ADMIN` / `USUARIO`).
- `toString()` muestra estos datos para listar por consola, pero deliberadamente **no expone la contraseña**.

VALIDACIONES

- Mail no vacío y único entre los usuarios activos.
- Editar/eliminar usuario inexistente → `EntidadNoEncontradaException` .

MENÚ

- Listar, crear, editar (revalidando unicidad si se cambia el mail).
- Eliminar con baja lógica, confirmación (S/N) y reintento en cada entrada numérica.

Un usuario eliminado no aparece más en los listados ni puede usarse para crear nuevos pedidos, pero sus pedidos anteriores siguen consultables para no perder el historial.

README.md

Documentación del repositorio del proyecto

REQUISITOS

Java 21

Versión mínima necesaria para compilar y ejecutar el proyecto.

EJECUCIÓN

NetBeans o terminal

Instrucciones de compilación y ejecución por ambas vías.

ESTRUCTURA

Paquetes

Organización del código por capas (modelos, repos, services, UI).

Incluye además los enlaces al **video de presentación**, al **informe PDF** y al **diagrama UML** del sistema.

Gestión de Pedidos — Modelado

Pedido · DetallePedido · interfaz Calculable

MODELO DE ENTIDADES Y COMPORTAMIENTO

- **Pedido** & **DetallePedido**: relación de **agregación** — el Pedido agrupa una lista de detalles, maneja estados y formas de pago, y se relaciona con un cliente.
- Interfaz **Calculable**: implementación de **calcularTotal()**, que usa **streams** para sumar los subtotales de forma eficiente.

ENCAPSULAMIENTO DE LÓGICA

- **addDetallePedido()**: validación de la cantidad y cálculo de subtotales.
- **findDetallePedidoByProducto()** & **deleteDetallePedidoByProducto()**: gestión segura del carrito antes de confirmar el pedido.

PedidoRepository y PedidoService

Capa de persistencia y lógica transaccional de pedidos

CAPA DE PERSISTENCIA

- `guardar()`, `buscarPorId()`, `listarTodos()`.
- **Auditoría:** `eliminar()` mediante baja lógica (flag/bandera) — no borra el objeto.

LÓGICA TRANSACCIONAL

- **Integración de Servicios:** al crear un pedido, coordina con `ProductoService` (stock) y `UsuarioService` (existencia del usuario).
- Bloque `try-catch` para revertir el stock ante fallos.

MenuPedido

Conexión de la lógica de negocio con la interfaz de usuario

- **Consola interactiva:** uso de `Scanner` para procesar las entradas.
- **Experiencia de Usuario (UX):** métodos `LeerEntero()` y `LeerLong()` con bloque try-catch para el control de datos inválidos.

Conclusiones

Lo que construimos como equipo

DISEÑO

Arquitectura en capas

Paquetes desacoplados: modelos, excepciones, repositorios, servicios y UI en consola.

CONSISTENCIA

Baja lógica en todos los módulos

Categorías, Productos, Usuarios y Pedidos preservan su historial en memoria.

ROBUSTEZ

Validaciones y excepciones propias

Manejo controlado de errores en cada capa de negocio del sistema.

Equipo Food Store: Alfredo Castillo (Comisión 3) · Cesia Castilla (Comisión 5) · Rocío Agüero (Comisión 6) — Programación II, UTN. ¡Gracias!